

Name : Verren Angelina Saputra

Applied Role : AI Solution Engineer

Technical Test Shopee - Engineering Knowledge AI Agent

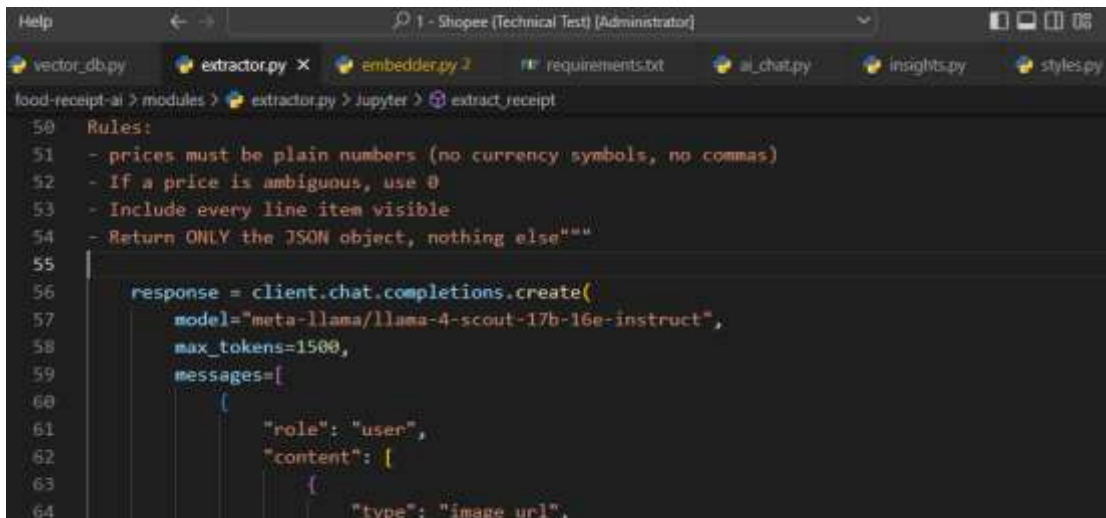
1. Describe differences between REST API, MCP in the context of AI.

Answer:

REST API, or Representational State Transfer API, is a communication standard between systems that has existed for a long time and is widely used by developers. Its mechanism is simple: the client sends an HTTP request to the server, and the server responds with a response, usually in JSON format. Meanwhile, **MCP**, or Model Context Protocol, is a newer protocol created by Anthropic in 2024, specifically designed so AI/LLMs can interact with tools and data dynamically in a two-way manner. That is why REST API has stateless characteristics, while MCP has stateful characteristics. Although MCP has more advanced capabilities than REST API, they are not competitors because MCP often wraps REST APIs behind the scenes to deliver data to AI. Below are the differences between them:

No	Characteristic	REST API	MCP
1	Protocol	HTTP / HTTPS	JSON-RPC 2.0 (via stdio / SSE / HTTP)
2	State	Stateless	Stateful
3	Connection Type	Open-close per request	Persistent session (one connection, many calls)
4	Communication Direction	One-way, client always initiates	Two-way, server can push notifications to AI
5	Suitable For	Web/mobile apps, system integrations, public APIs	AI agents, LLM workflows, Claude Desktop integration
6	Discovery endpoint	Developer must read documentation	Automatic because AI can ask "what tools are available?"

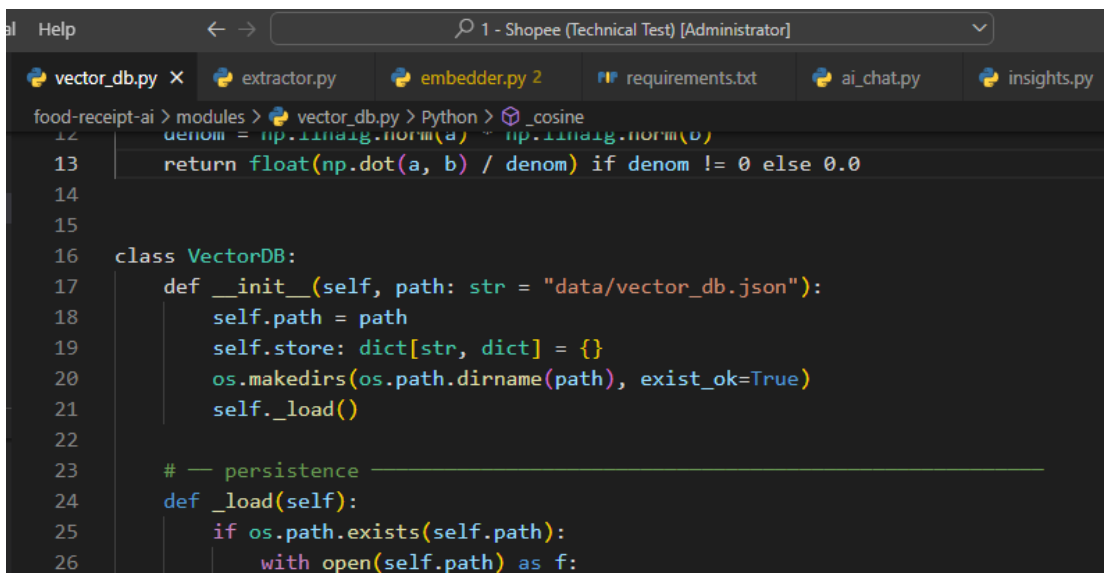
An example of implementation can be seen in the automation system I built called "VRECEIPT TRACKER", where there are several uses of REST API:



The screenshot shows a Jupyter Notebook interface with a browser window at the top displaying '1 - Shopee (Technical Test) [Administrator]'. The notebook has several tabs: 'vector_db.py', 'extractor.py', 'embedder.py', 'requirements.txt', 'ai_chat.py', 'insights.py', and 'styles.py'. The active cell is in 'extractor.py' and contains the following Python code:

```
50 Rules:
51 - prices must be plain numbers (no currency symbols, no commas)
52 - If a price is ambiguous, use 0
53 - Include every line item visible
54 - Return ONLY the JSON object, nothing else"""
55
56 response = client.chat.completions.create(
57     model="meta-llama/llama-4-scout-17b-16e-instruct",
58     max_tokens=1500,
59     messages=[
60         [
61             "role": "user",
62             "content": [
63                 {
64                     "type": "image_url",
```

This is a normal HTTP REST call to the Groq server. Every call is a new request, stateless, and has no persistent connection.



The screenshot shows a Jupyter Notebook interface with a browser window at the top displaying '1 - Shopee (Technical Test) [Administrator]'. The notebook has several tabs: 'vector_db.py', 'extractor.py', 'embedder.py', 'requirements.txt', 'ai_chat.py', and 'insights.py'. The active cell is in 'vector_db.py' and contains the following Python code:

```
12 denom = np.linalg.norm(a) * np.linalg.norm(b)
13 return float(np.dot(a, b) / denom) if denom != 0 else 0.0
14
15
16 class VectorDB:
17     def __init__(self, path: str = "data/vector_db.json"):
18         self.path = path
19         self.store: dict[str, dict] = {}
20         os.makedirs(os.path.dirname(path), exist_ok=True)
21         self._load()
22
23     # — persistence —————
24     def _load(self):
25         if os.path.exists(self.path):
26             with open(self.path) as f:
```

This is only a local JSON file read/written manually. It is not an MCP server, not any protocol, and purely just a normal Python class.

```

26
27 def _sentence_transformer_embed(text: str) -> np.ndarray:
28     global _model
29     from sentence_transformers import SentenceTransformer
30
31     if _model is None:
32         # ~80MB, downloads once and caches in ~/.cache/huggingface/
33         _model = SentenceTransformer("all-MiniLM-L6-v2")
34
35     vec = _model.encode(text, normalize_embeddings=True)
36     return vec.astype(np.float32)
37
38
39 # — Fallback: keyword hash vector (256-dim, no extra deps) —————
40 def fallback_embed(text: str) -> np.ndarray:

```

This is a local library running on the same machine, not any communication protocol. Everything is request-response only, with no tool discovery, no persistent session, and no bidirectional communication.

2. How REST API, MCP, can improve the AI use case.

Answer:

Both play important roles in improving AI use cases. First, REST APIs allow AI to connect to external data and services in real time. Without REST APIs, AI could only rely on its training data, which may already be outdated. For example, for accessing real-time stock prices, REST APIs give AI access to real-time data, and AI can also perform real actions such as sending emails, processing payments, and updating databases, all integrated into existing systems without rebuilding from scratch.

Meanwhile, MCP takes AI capability to a much more advanced level. Instead of simply “answering questions,” it enables AI to complete multi-step tasks independently. As a result, workflows that previously had to be manually handled through REST APIs can now become automated because AI acts like an assistant that understands work context, not just answering one question but completing an entire task from start to finish autonomously. Examples can be seen below:

No	Use Case	REST API	MCP	AI improvement
1	Customer Service Chatbot	Access data or APIs related to orders,	Automate multi-step workflows: check → process → notify in one	Handling time is significantly reduced

		shipping status, and customer history in real time. • Process transactions such as refunds and status updates via POST/PATCH. • Integrate with existing systems without rebuilding them.	session, so the AI understands what actions it can perform. • Conversation context is stored, giving the AI stateful characteristics.	because cases can be resolved without human agents, leading to improved customer satisfaction
2	AI Coding Assistant	• GitHub/GitLab APIs such as reading files, committing code, and creating public repositories. • Trigger CI/CD pipelines and check the results. • Fetch documentation and package registries.	• AI can plan and execute multi-step tasks without intervention, and it has persistence capability (remembering previous test history). • Each created function such as <code>read_file</code>, <code>write_file</code>, <code>run_tests</code>, and <code>git_commit</code> is used as a tool.	Developer work becomes easier because they can focus on architecture instead of repetitive tasks that usually take hours. End-to-end bug fixing becomes much faster with minimal developer intervention.
3	AI Scheduling	• Can access Google Calendar API	• AI can handle the entire workflow	Eliminates back-and-forth

	g Assistant	and Zoom/Meet API, but AI does not know anyone's schedule and can only suggest general available times.	including edge cases such as declines, timezone differences, and conflicts. AI uses functions as tools such as check_availability, create_event, create_zoom, and send_invitation, allowing it to create events or send invitations autonomously.	scheduling emails, so meetings can be arranged in one short conversation. Makes it easier to handle timezone differences and user preferences automatically.
--	-------------	---	--	--

3. How do you ensure that your AI agent answers correctly?

Answer:

There are many ways to ensure that an AI agent answers accurately. I will explain based on my technical experience when building my thesis project, an LLM chatbot using a hybrid architecture combining RAG and function calling.

- Evaluation and Testing:

One way to prove AI correctness is by measuring it systematically, not just based on "feeling correct." For example, we split the data for testing and prepare correct ground-truth answers. Then we calculate how many AI answers match those answers. We can also use semantic similarity by applying embeddings to check whether the meaning of the answer matches or not. Evaluation should be run every time prompts or models are changed, similar to unit testing. At the production monitoring level, we can use A/B testing to compare Prompt A vs Prompt B on real traffic.

I had experience during my thesis where function-calling accuracy was measured by comparing tax calculation outputs against answer keys based on the original formula. This was used to determine whether function calling could accurately process calculation-related questions. The result was 9 out of 10 questions answered correctly, or 90% accuracy.

- **Human-in-the-Loop:**

For high-risk decisions, AI should not work alone. Humans must remain involved in the review process. I gather experts in the related field and conduct User Acceptance Testing (UAT) together. The goal is to detect answer anomalies where results differ significantly from baseline expectations. Users must explicitly confirm before AI proceeds. Every review is recorded as an audit trail, then categorized into: PASS, RETEST, BLOCKED, and FAILED.

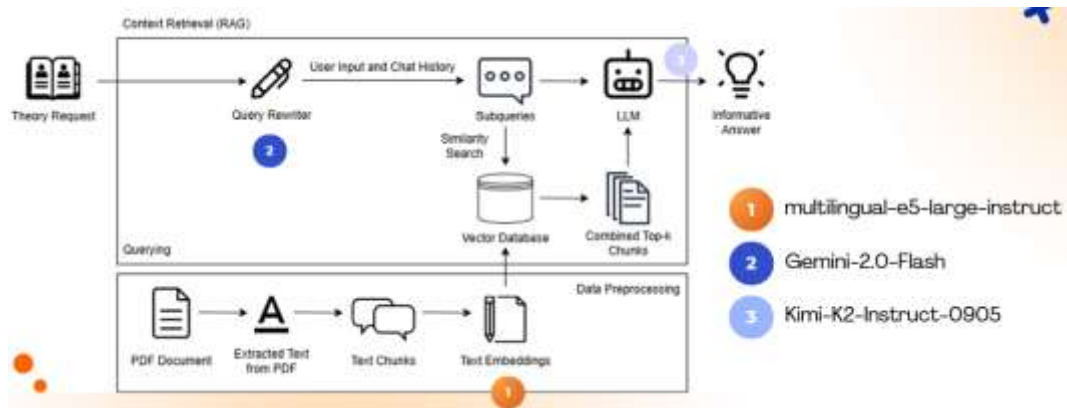
The screenshot displays a detailed User Acceptance Testing (UAT) spreadsheet. It includes a header section with project details, a table of test cases with columns for Test Case ID, Method, Test Case, Pre-Conditions, Steps to Execute, Test Data, Expected Result, Actual Result, Status, and Notes. The test cases are categorized into 'Basic Functionality and UI Fundamentals' and 'Scenarios Regarding PPH's Tax Calculation'. The spreadsheet shows various test results, including 'Running well', 'Blocked', and 'Failed'. A summary table at the bottom right provides an overview of the test results, including the number of tests passed, failed, and blocked.

Test Case ID	Method	Test Case	Pre-Conditions	Steps to Execute	Test Data	Expected Result	Actual Result	Status	Notes / Comments
TC001	Homepage	Visit homepage website	Internet connection	1. Open browser 2. Go to Website URL	NA	Website loads and content is displayed correctly	Website loaded successfully	PASS	Running well
TC002	Main Buttons	Click "Learn More" button	Website loaded	Click "Learn More" button	Button click	Clicked response with navigation about general tax matters	Navigation shown	PASS	Running well
TC003	Main Buttons	Click "Hiring Paper" button	Website loaded	Click "Hiring Paper" button	Button click	Clicked data entered into an calculator for calculation	Calculation flow initiated	PASS	Running well
TC004	Main Buttons	Click "Care Partnership" button	Website loaded	Click "Care Partnership" button	Button click	Clicked response flow to only view the available methods	Click display step shown	PASS	Running well
TC005	Default Response	Handle empty input on input field	Website loaded	1. Click "Hiring Paper" 2. Input empty value or remove the calculation without click any buttons	NA	A. User calculates tax in cases "Hiring Paper" B. But should "Click Hire Paper" or similar response	Calculation flow initiated	PASS	Previously state for the input field and calculation. But now state is changed. But should be changed. But should be changed. But should be changed.

- **RAG & Grounding:**

Retrieval-Augmented Generation ensures AI does not hallucinate but answers based on real injected context. This makes the answers: More factual because they are based on real documents and updatable because the data can always be refreshed.

In my thesis chatbot project, I also used RAG and Function Calling for a tax law expert chatbot, where AI answers only based on the provided context stored in vector DB using semantic search, not from training memory.



Then evaluation was done using RAGAS with its main metrics:

- (1) **Faithfulness** → Measures how much claims in the answer can be traced back to retrieved documents.
- (2) **Correctness** → Measures answer accuracy compared to ground truth.
- (3) **Precision** → Measures how relevant retrieved documents are.
- (4) **Recall** → Measures how complete the retrieval is.
- (5) **Answer Relevancy** → Measures how directly the AI answers the user's question.

Chatbot	Faithfulness	Correctness	Precision	Recall	Answer Relevancy
BeeTax	0.71	0.61	0.79	0.79	0.81

- **Prompt Engineering:**

In my thesis, prompt engineering was very useful because answer quality depends heavily on how AI is instructed. Explicit system prompts were designed to define: AI role, limitations, and output format.

For example, we provide context that AI should act as a tax expert system capable of understanding tax terminology and determining whether a question is theoretical or calculation-based. We can also provide 2–3 examples of correct input/output so AI learns to reason step by step before answering. This method is zero-cost and one of the easiest to implement.

Below is a code snippet from my thesis project chatbot called BeeTax.



```
def retrieve_docs(query):
    return faiss_db.similarity_search(query)

def get_context(documents):
    context = "\n\n".join([doc.page_content for doc in documents])
    return context

#Step3: Answer Question

▼ custom_prompt_template = """
Use the pieces of information provided in the context to answer user's question.
If you dont know the answer, just say that you dont know, dont try to make up an answer
Dont provide anything out of the given context
Question: {question}
Context: {context}
Answer:
"""
```

4. Describe what can you do with Docker / Containerize environment in the context of AI

Answer:

Docker transforms AI from just an experiment in a notebook into a product that can be deployed, scaled, and maintained professionally. Based on my experience, Docker helps solve several important problems:

- **Library Versioning and Environment Differences** = When I worked on university projects with my teammates, Docker helped solve issues related to different versions of CUDA, Python, or other libraries. Docker freezes the entire environment into one isolated unit, so each AI service has its own dependencies. My laptop uses Python version 3.11, while my teammate still uses Python 3.10, and initially some libraries were not compatible. However, when we use a Dockerfile, the environment becomes identical for everyone without exception.
- **Packaging the Entire AI Application Stack** = Each component such as the app, database, and model server can be placed in separate containers, and there is one file called docker-compose.yml that orchestrates all services in a single configuration. We do not need to repeatedly install requirements because Docker layer caching allows us to rebuild only when new dependencies are added. In addition, Docker allows us to store API keys using environment variables instead of hardcoding them in files. This is very useful for improving AI security.

- **Reproducible Model Training Pipeline** = To keep experiments reliable, everyone can follow the same training workflow from data preparation to model export through the training pipeline. We can create multiple stages, and in my experience this pipeline can also be connected with MLflow. While the training pipeline is running, MLflow works in parallel to track experiment results, record hyperparameters, metrics, output weights, and determine which models are used in production through model registry. When I used Docker, if the code had an error, the pipeline would stop immediately and Docker would provide error information shown through GitHub Actions. This makes deployment much safer because it ensures that only stable and tested code is deployed to production, not code that still contains bugs.
- **Advantages in Model Serving** = I also learned that AI models which originally could only run manually through scripts can be transformed into always-active services that can be accessed anytime by other applications through HTTP requests, because the model weights, preprocessing, and post-processing are stored inside the container. If the model version changes, code usually becomes incompatible or breaks, but Docker supports rolling updates where requests can still be served during the update process. For example, when I was working on the Monelytics machine learning project, after upgrading Python from version 3.9 to 3.11, Streamlit became incompatible and I had to downgrade manually. However, with Docker and containerization, this problem no longer needs to be a concern.

5. How do you finetune the LLM model from raw ?

Answer:

Fine-tuning an LLM is the process of retraining a pre-trained language model using specific data so that the model can perform much better for a particular task. I often do this, especially when working on the Sounify project to process audio input with English accents from different countries and produce output that detects the country of origin based on its unique accent. Usually, there are several steps I follow:

- (1) **Raw Data Preparation** = First, I collect data with a specific domain based on the model's objective, for example additional audio datasets from various country accents. After that, I perform preprocessing such as removing noise, deleting corrupt or duplicate data, normalizing audio file formats, and ensuring the data quality remains consistent so the model does not learn from poor-quality data.
- (2) **Data Labeling and Formatting** = Since the data used is audio, before training I usually convert it into numerical representations such as spectrograms, MFCC, or audio embeddings so it can be processed by the model. In the context of text-based LLMs, this stage is similar to tokenization.

- (3) **Training / Fine-Tuning the Model** = Then, the pre-trained model is retrained using the dataset by performing a forward pass, calculating the loss or error compared to the correct labels, and then doing backpropagation to update the model weights. At this stage, I also adjust hyperparameters such as learning rate, batch size, and number of epochs so the training can run optimally.
- (4) **Evaluation and Validation** = After the training is complete, I evaluate the model using a validation or test set to check whether the model has actually improved. The metrics I usually review include accuracy, precision, recall, or confusion matrix depending on the use case.
- (5) **Deployment and Monitoring** = If the performance meets the target, the model is deployed to the production environment for users to use. After deployment, the model's performance is continuously monitored so that if model drift or performance degradation happens, retraining can be done again.